

BSc (Hons) in **Computer Science****SE3062 : Intelligent Systems**

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing**Practical 01**

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation**Task 0 : Setting up and getting the starter Code**

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance measure, Environment, Actuators, and Sensors.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

food_positions, walls, width, height.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

`self.opponents` starts empty and is filled only if `num_opponents > 0`. The program explicitly sets `num_opponents=0`, so the while-loop never runs and `self.opponents` stays an empty list. Because the environment creates zero opponents, only the main agent (`self.agent_pos`) acts.

Result: The baseline environment is a Single-Agent scenario.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

The complete history of all sensory inputs or data an AI agent has ever received from its environment up to the present moment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors component

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

The environment is Partially Observable.

agent can see: Its own current [x, y] location, Whether food is on its current tile, Whether it bumped into a wall on its current step.

agent cannot see: the list of coordinates where food is located (self.food_positions). It only knows if food is under its feet, It does not receive the coordinates of walls, It does not know the full map layout.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

External Results measure task outcomes and environmental interaction (e.g., `self.score -= 5` when colliding with `self.walls` in `grid_game.py`). This ensures the agent is judged strictly on safety and goal completion.

Internal Processing Effort measures computational cost. While useful for efficiency profiling, using it as a performance metric fails to measure whether the agent actually succeeded in its task.

Measuring external environment results (such as wall collisions or food collection) evaluates what the agent accomplishes in the world, whereas measuring internal processing effort (such as CPU time or search iterations) evaluates how hard the agent thinks. Structurally, the PEAS Performance Measure must rely on external state changes because it provides an objective, standardized benchmark of task success and safety across different agent architectures. Relying on internal computational effort instead would break the decoupling between agent logic and environment rules, while inviting metric exploitation such as an agent choosing to do nothing simply to minimize compute costs rather than solving the actual task.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Adding `self.toxic_traps` to the environment while omitting it from `get_percept()` makes the environment Partially Observable. Because the environment's true state includes trap locations that the agent's sensors cannot detect, the agent receives incomplete state information, forcing it to make decisions under uncertainty.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.
10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding 'smells_toxin' expands the agent's percept sequence with critical environmental feedback. Rationality requires an agent to choose actions that maximize expected performance measure given its percepts. By providing sensory awareness of toxins, the agent can compute expected utility by balancing positive rewards against negative trap penalties—and select optimal actions to avoid damage and maximize its cumulative score.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic Vacuum World Exploit illustrated in Lecture 01 was that the agent was rewarded +1 point for every clean tile. Instead of cleaning all tiles and stopping, the agent discovered it could repeatedly dirty a tile and re-clean it in an infinite loop, accumulating unlimited points without ever achieving the actual goal. This happened because the metric rewarded the action of cleaning rather than the desired world state of being fully clean, allowing the agent to exploit the faulty measure rather than solving the task.

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!

